# Probabilistic multistart with path relinking for solving the unconstrained binary quadratic problem

Mark Lewis*

Craig School of Business,
Missouri Western State University,
4525 Downs Dr., Saint Joseph, MO, 64507, USA
E-mail: mlewis14@missouriwestern.edu
*Corresponding author

Gary Kochenberger

School of Business,
University of Colorado,
1250 14th St., Denver, CO, 80202, USA
E-mail: Gary.Kochenberger@cudenver.edu

**Abstract:** The unconstrained binary quadratic problem (UBQP) has been shown to be an excellent framework from which to solve many types of problems, both constrained and unconstrained. In this paper we investigate a solution technique for UBQP that is based on perturbing a solution by drawing from the distribution of variables' estimated effect as determined via an unbiased design of experiments (DOE) sampling of the solution space. Solution perturbation is followed by a steepest ascent local search with path relinking. A simple implementation on benchmark problems compares well in time and solution quality with published results on benchmark problems of size up to 7,000 variables. A new set of larger problems having up to 15,000 variables and with non-uniform magnitude distributions of the elements in Q are also investigated and provide evidence that magnitude distributions of Q values affect problem difficulty. These large difficult problem instances required a more sophisticated path relinking approach as well as dynamic adjustments to perturbation sampling.

**Keywords:** multi-start heuristics; path relinking; binary quadratic program; preprocessing; local search; experimental design; benchmark problems; solution perturbation.

**Reference** to this paper should be made as follows: Lewis, M. and Kochenberger, G. (2016) 'Probabilistic multistart with path relinking for solving the unconstrained binary quadratic problem', *Int. J. Operational Research*, Vol. 26, No. 1, pp.13–33.

**Biographical notes:** Mark Lewis has a BS in Electrical Engineering from the University of Kansas and worked as an Engineer in Avionic Development at Lockheed Martin for 12 years before getting a PhD in Operations Research from Southern Methodist University in Dallas, Texas, USA. He is currently a Professor teaching information systems and operations management. He has published in *International Journal of Operational Research*, *INFORMS Journal on Computing*, *Computers and Operations Research*, *Operations Research Letters*, *Networks*, *European Journal of Operational Research* and others.

Copyright © 2016 Inderscience Enterprises Ltd.

Gary Kochenberger's academic specialty concerns building, testing, and implementing algorithms for solving resource allocation problems that arise in both the private and public sectors. In recent years, his focus has been on problems of a combinatorial nature. He has published four books and numerous research papers in such journals as the *Operational Research Management Science*, *Mathematical Programming*, *International Journal of Operational Research*, *Journal of Optimization Theory and Applications*, *Operation Research*, *Computers and Operations Research*, *Naval Research Logistics Quarterly*, *Decision Sciences*, *Journal of the Operational Research Society*, *European Journal of OR*, *Interfaces*, *Operations Research Letters*, *Omega*, and the *Journal of the Production and Operations Management Society*.

## 1 Introduction

The unconstrained binary quadratic problem (UBQP) sometimes referred to as $xQx$, has been shown to be an excellent modelling framework for a wide range of problems. In addition, constrained problems can be transformed to unconstrained via objective function penalties, as presented in Hansen et al. (1993) and Kochenberger et al. (2004). The only difference in the wide variety of problems modelled as UBQP is the relative magnitude and density distribution of the linear and quadratic elements of the Q matrix. Thus, a presolve analysis of the Q matrix should be a useful guide to a search algorithm.

In this paper we demonstrate that UBQP's binary, unconstrained framework provides an excellent match to experimental design sampling techniques that are based on estimating the effects of yes/no type decisions. Results of the sampling are used as a global guide to influence local search as well as solution perturbation during optimisation. In this paper, we focus on demonstrating the benefits of a $Q$ analysis prior to search by comparing two presolve techniques, one based simply on using the mean and standard deviation of $Q$, the other using design of experiment (DOE) methodologies that are scaled to address very large problem instances, as described in Lewis (2008) and Lewis et al. (2006). The information from the preprocessing is used to generate probabilities to perturb components of a solution during iterated solution improvement and to influence variable selection during the solution improvement phase as well as path relinking. To clarify the implementation, detailed pseudo code is presented and discussed.

The rest of the paper is organised as follows. A literature survey of UBQP is Section 2, followed by discussion of the algorithms compared in Section 3. Results from two sets of benchmark problems plus an additional set of new problems is presented and discussed in Section 4. Section 5 discusses the suitability of the probability assignment assumptions made by presenting detailed results of some DOE testing of problems in Section 3. Section 6 provides a summary and conclusions as well as ideas for future research.

## 2 Literature

The unconstrained binary quadratic programming (UBQP) problem is described as

$$\max\ x^t Q x\ x \in S$$

where $S$ represents the binary discrete set $\{0, 1\}^n$ or $\{-1, 1\}^n$ and Q is an $n$-by-$n$ square symmetric matrix of real or integer coefficients where the diagonal of Q is referred to as the linear elements and the off-diagonal the quadratic. With this simple construct a wide range of combinatorial optimisation applications can be modelled and solved.

Early work with UBQP by Barahona et al. (1988) dealt with the physics problem of finding ground states of spin glasses with exterior magnetic fields. Subsequently, published UBQP applications include max-cut (Boros and Hammer, 1991; Kochenberger et al., 2013), machine scheduling (Alidaee et al., 1994), maximum clique (Pardalos and Xue, 1994), number partitioning (Alidaee et al., 2005) and max 2-sat (Kochenberger et al., 2005a).

The applicability of UBQP is greatly extended by the ability to model constraints with quadratic infeasibility penalties in the objective function (Kochenberger et al., 2004). Via these penalty functions a wider net is cast over the range of problems that can be solved within the UBQP framework. With this approach, Lewis et al. (2005) solve the problem of assigning tasks to processors in a distributed, multitasking computer environment such that the sum of the resultant task completion costs (linear, diagonal Q elements) and inter-task communication costs (quadratic portion of Q) are minimised. The constraints requiring each task be assigned to one processor are transformed to quadratic penalties in the objective function. In addition, Kochenberger (2005b) use this quadratic penalty approach for clustering microarray data and Wang et al. (2006) group machines and parts in a flexible manufacturing environment. It is used to solve set partitioning problems in Lewis et al. (2008) and for set packing in Alidaee et al. (2008). The linear ordering problem is transformed to UBQP in Lewis et al. (2009). Other applications include sum colouring (Douiri and Elbernoussi, 2012), robust graph colouring (Wang and Xu, 2013) and capacitated clustering (Lewis et al., 2014).

Exact solution methods for UBQP often incorporate branch-and-bound (for example, Gulati et al., 1984; Pardalos and Rodgers, 1990; Billionnet and Sutter, 1994; Palubeckis, 1995; Huang et al., 2006). In addition, commercial solvers such as Cplex and Gurobi are also capable of optimising problems with quadratic objective functions. Other exact methods that are not based upon branch-and-bound include work by Chinchuluum et al. (2005), Pan et al. (2008), Gueye and Michelon (2009), Dinh et al. (2010) and Mauri and Lorena (2012).

Because UBQP is NP-hard, large problem instances require heuristics with a large body of published literature on the subject. The research often involves an implementation of either tabu search, simulated annealing, evolutionary algorithms, local search, path relinking or various combinations and permutations of these. Suggested selected reference papers are Glover et al. (1998) who propose an adaptive memory tabu search algorithm, incorporating strategic oscillation around critical events (Amini et al., 1999), a scatter search approach to UBQP, one-pass heuristics (Glover et al., 2002), and a hybrid genetic tabu search with multi-parent crossover (Lu et al., 2010b).

Approaches that start with a random population, such as the memetic algorithm in Lu et al. (2010a) might benefit from the techniques described in this paper for creating diverse solution sets and the DOE preprocessing might also be useful in discovering memes. The framework of balancing intensification and diversification presented in Lu et al. (2010a) supports the probability oscillation techniques employed in this paper. The probabilistic GRASP-tabu for UBQP discussed in Wang et al. (2013b) constructs solutions via a sorted order of the variables' 'objective function increment' value and

they implement the principle of strongly determined variables to reconstruct new solutions by strategic oscillation (the construction/destruction of parts of a selected solution). Perturbing solutions by leveraging a sorted order of effects, and modifying parts of a solution are also ideas that were implemented in concept in this paper.

The simple search algorithm presented in this paper was developed to highlight the effects of preprocessing the Q matrix and the local search is based upon iteratively selecting one variable with the largest effect [referred to as 1-flip, see for example Glover et al. (2002) or Merz and Freisleben (2002)] until no improvement is available. The algorithm presented here is similar to the iterated tabu search algorithm presented in Palubeckis (2006), although we do not incorporate a tabu search component. We also borrow preprocessing ideas from the work of Boros et al. (2007) and the landscape analysis of Merz (2004). Various multistart ideas from Palubeckis (2004) were examined and that paper is the basis for the second set of benchmark problems up to size 7,000 variables. Since the path relinking work by Wang et al. (2012) uses similar computing power and finds the best known solutions for the benchmark problems, we compare our results to theirs.

The main contributions of this paper are the benefits of an experimentally designed sampling of the solution space and the effects that the relative magnitudes of the Q elements have on problem difficulty.

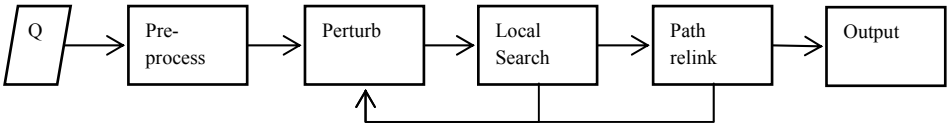
## 3 Algorithm description

In this paper we present a basic algorithm used to demonstrate the benefits of preprocessing the Q matrix and in Section 4.1 present the performance modifications made for the larger, more diverse problems studied. The main algorithm flow is shown in Figure 1 and starts with a preprocessing that as a baseline simply computes the average and standard deviation of the elements of Q, which is compared to an unbiased experimental design techniques to generate an estimate of each variable's average effect on the objective function. The term 'unbiased' is used with reservation because an assumption is made that large numbers of variables are not interacting in a way that dramatically affect the objective function value. Rather, we assume variables tend to operate individually or in small clusters each having at least one individually significant component. Since UBQP is NP-hard, this assumption may not always be valid, but was shown to be useful in this research. The preprocessing results are used to influence solution perturbation (line 6) and variable selection in local search and path relinking (lines 7, 11, 15) of Figure 1.

The initial starting solution input to the Perturb component may be any random or set starting solution. In this proof of concept implementation, we started with a solution of all 1s to be perturbed and then input to local search component. Local search is a greedy 1-flip search that repeatedly selects the single changing variable that has the greatest positive effect on the maximising objective function. This solution is then fed back into the main loop. To isolate the effects of the DOE presolve, three approaches were coded and tested on two sets of benchmark problems. These three approaches are G for greedy

only search, GR for greedy plus path relinking, DG which adds the presolve to the greedy approach and DGR which adds path relinking to DG.

**Figure 1** Main algorithm pseudocode




```
1. Inputs:  Q matrix, n
2.  Output:  Binary vector x and its objective function value
3.  DOE_Sampling_Presolve ( Q )
4.  Initialize_starting_solution ( x )
5.  While (iterations < max_iterations ) {
6.    Perturb_Solution ( x )
7.    repeatdelta = Local_Search ( Q,x )until( delta== 0 )
8.    if ( obj_value ( x ) > best_obj ) {
9.          best_obj = obj_value
10.         if ( number_difference_bits ( x, best_x ) > limit )
11.                Perform path relinking between x and best_x
12.               Save_solution_for_path_relinking ( x)
13.    }  // end if new best_obj
14. } // end main while loop
15. Perform path relinking between best_x and all saved solutions
```

A detailed description and application of using DOE in optimisation is presented in Lewis (2008) and the technique used in this paper is summarised in the Appendix. Briefly, the DOE creates a diverse set of solutions whose various effects on the objective function are recorded and analysed in aggregate so that vital statistics, such as average effect of a variable $i$ on the objective function, average effect of all variables, standard deviations of average effects and so on can be computed. The pseudo-code is presented in Figure 2.

In the Perturbation component of the main algorithm at line 6, a purely random $x_i$ perturbation is replaced with one that is based on the DOE estimated effect of $x_i$ on the objective function. Thus, if the DOE indicates that a variable $x_i$ when set to 1, tended to have a negative effect on the objective, then the probability that $x_i = 1$ is closer 0. If the DOE indicates that a variable has a strong positive effect, then $1 > p(x_i = 1) > 0.5$ and $x_i$ will tend to be perturbed to a 1. Another assumption of DOE known as 'sparsity of variable effects', similar to the Pareto principle, claims that most variables will have little or no effect. Thus, DOE tries to find those variables that do have strong negative or positive effects and are identified as statistical outliers from the average effect. The DOE effect of each $x_i$ is stored in the `DOE_estimates` array. Note that this approach does allow a variable with a strong positive effect to be set to 0 (and vice-versa) at a small probability over the course of the thousands of iterations of the Perturbation ←→ Local Search loop.

**Figure 2** DOE presolve module

```
1. Inputs:  Q, DOE_settings_file, num_tests
2.    Output:  array of estimated effects of switching a bit from 0 to 1
3.    for ( i = 0 to num_tests) {
4.    x = Read_DOE_settings_at_test_num (DOE_settings_file, i )
5.          DOE_result[ i ] = GetObjectiveValue ( x, Q )
6.          }  // end for
7.     DOE_estimates ← CalculateEstimates ( DOE_result )
```

Basic perturbation of a solution used in the GR approach is performed by selecting at random a 0 or 1. If the randomly generated value is a perturbation to the current $x_i$ and if this local change exceeds an aspiration level, then the perturbation is made. The aspiration level is based on the statistical analysis of Q and was generally set to be about 0.5 of a standard deviation below the average effect. In other words, `if(delta_f > avgVarEffect - std_dev * std_dev_divider` then the perturbation is accepted. The average and standard deviation of the elements of the Q matrix are used for this basic approach. An analysis of the two perturbation methods is presented in Section 5.

**Figure 3** Basic perturbation

```
1. Inputs:  x, n
2.   Output:  x
3.   for ( i = 0 to n ) {
4.   j = Get_Random_Int( 0, 1 )
5.   if ( x[ i ] != jAND Delta_objective( x, i ) > aspiration_level )
6.              x[ i ] = j  // make the change
7.    }  // end for all variables i in x
```

Figure 4 shows the perturbation method based on using the DOE estimates for each $x_i$. We found that the order in which the $x_i$ are examined affect the quality of the overall perturbation of the solution $x$ and use the ordering of estimated effects in the perturbation loop. For example, suppose the *aspiration level* = 7, flipping $x_1$ yields a *delta* = 5, flipping $x_2$ yields a *delta* = 10, but flipping both yields a *delta* = 25. If $x_1$is examined first it will not be flipped because it does not exceed our aspiration level and the *delta* = 25 will not be found, but it will be found if $x_2$ is examined first. Thus, $x_i$ were examined from largest estimated effect to smallest. After analysing the p($x_i$) will have an effect greater than the average effect and discovering that in most cases the probabilities clustered close to 0.5, we examined the distribution of DOE effect estimates and found them to be more widely distributed. In fact, as shown in Section 5, the effects were often normally distributed, which leads to assigning probabilities based on standard ideas from the normal distribution. For example, $x_i$ with DOE effects more than three standard deviations from the average were given a high probability of being perturbed to 1. Testing showed that a combination of the two probability assignment approaches gave the best results.

**Figure 4** Initial DOE-based perturbation

```
1.    Inputs:  x, n, DOE_estimate, DOE_sort_order
2.   Output:  x
3.   for ( i = 0 to n ) {
4.   j = DOE_sort_order ( i )
5.          if ( DOE_estimate[ j ] > u + 3*s ) prob = 0.95
6.          else if (DOE_estimate[ j ] > u + 2*s ) prob = 0.8
7.          else if (DOE_estimate[ j ] > u + s ) prob = 0.7
8.          else if (DOE_estimate[ j ] < u - 3*s ) prob = 0.05
9.          else if (DOE_estimate[ j ] < u - 2*s ) prob = 0.25
10.         elseif (DOE_estimate[ j ] < u - s ) prob = 0.35
11.         else prob = 0. 5
12.         if ( prob > Get_random_float (0,1) ) k = 1 else k = 0
13.  if ( x[ j ] != k AND Delta_objective( x, j ) > aspiration_level )
14.              x[ j ] = k  // make the change
15.   }  // end for all variables i in x
```

**Figure 5** Local search

```
1. Inputs:  Q matrix, n, x, DOE_estimate
2. Output:  Delta from starting objective function value
3. max_delta = 0
4. for ( i = 0 to n ) {
5.      delta = ComputeDeltaForVariable_i (Q, n, x, i)
6.      if (delta > max_delta )  {
7.            max_delta = delta
8.            best_index = i
9.           AddIndexToCandidateList( list, i )
10.         } // end if
11.      }   // end for
12.  i = SelectRandomFromCandidateList ( list )
13.  j = SelectRandomFromCandidateList ( list )
14.  if ( DOE_estimate[ i ] < DOE_estimate[ j ] ) i = j
15.  delta = ComputeDeltaForVariable_i (Q, x, i)
16.  if ( delta > percent_limit * max_delta )
17.      best_index = i
18.      max_delta = ComputeDeltaForVariable_i (Q, n, x, i)
19.      }
20.  return max_delta
```

The Local Search component pseudo-code is presented in Figure 5. The information from preprocessing is used for variable selection in routine in lines 12 and 13. The `Local_Search` routine examines the immediate effect of flipping $x_i$ and collects a candidate list of all variables with a beneficial effect. When the list is small enough, two solutions are randomly selected from the list and if the one with the best estimated effect

is within a percent of the best effect, then it flipped, as opposed to always selecting the variable with the largest effect. Thus, by combining a global effect with a local one, diversity is introduced into the search. The scope of the paper was limited to highlight these contributions. The search algorithm presented could reasonable be improved via modifications such as the long term memory structures described in Glover et al. (2010) and the backbone guided tabu search with variable fixing/freeing described in Wang et al. (2013a). However, methods of incorporating preprocessing techniques with adaptive memory structures and tabu search in order to improve solution quality and time is reserved for future research.

When a new best solution is found that is considered diverse from the previous solution, it is saved as one of three candidate solutions for path relinking. In our implementation, the diversity limit was defined as when the number of different bits was greater than a number in the range [5, 10]. Path relinking between the current solution and the current best solution, at every 1,000 iterations, also increases diversity between the solutions being linked. Finally, at the end of the main loop, prior to termination, path relinking between all saved high quality, diverse solutions is performed.

Path relinking is accomplished in a manner similar to the `Local_Search` routine. In the `Path_Relinking` routine, a list of difference indices between the starting and ending solution is created in line 3 and the bit with the largest positive effect is flipped in line 6, that bit is removed from the list, the size of the list is reduced in line 8, and the process is repeated until the starting solution has been converted to the ending, checking for improvements to the best solution along the way. The *`FlipBestBitFromDiffSet(solution1, num_diff, diff_indices)`* routine may include `DOE_estimate` data to influence the selection of the bits flipped along the path taken from the starting to the ending solution. Note that 'the largest positive effect' may actually be a negative effect during the linking operation. This last approach is labelled DGR when path relinking is incorporated with the other approaches using DOE preprocessing, else GR when using simple statistical preprocessing methods.

**Figure 6** Basic path relinking

```
1. Inputs:  Q, n, start_sol, end_sol, DOE_estimate, num_diff
2. Output:  any new best solutions
3. diff_indices ← ReturnListofDifferenceIndices(start_sol, end_sol)
4. num_bits_between_solutions = num_diff
5. while ( num_bits_between_solutions > 0 ) {
6.      delta_obj = FlipBestBitFromDiffSet(solution1, num_diff, diff_indices)
7.      if (delta_obj + obj_val > best_obj) RecordNewBestObjective
8.      num_diff–
9.      }  // end for
```

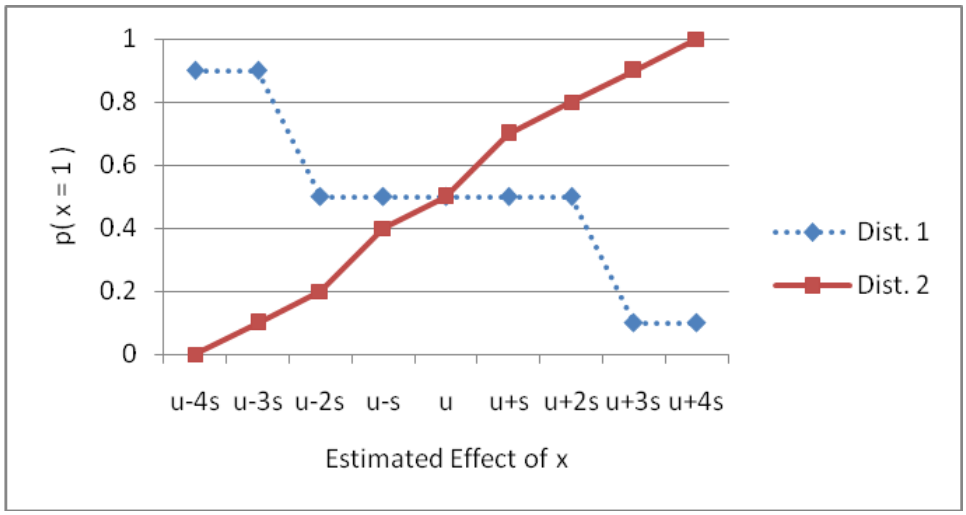
Important parameters are as a function of the size of Q or by information from the DOE presolve. e.g., the aspiration parameter for flipping a bit in the multi-start perturbation portion of the main loop (line 8) is based on the average estimated effect and some multiple of the standard deviation. In other words, `aspiration_level = avgVarEffect - std_dev * std_dev_multiplier`and if the delta for $x_i$ is greater than the aspiration level, then the perturbation is accepted.

The approach detailed above worked well for the benchmark problems, but for the larger, more diverse problems changes were made to the sampling and aspiration portions of the algorithm. These changes were deemed necessary after Wang et al. (2012) provided their executable program to use for comparison purposes, revealing that while our basic approach worked well enough on the previous benchmarks, it did not scale as well for the new set of problems, which are described in more detail in Section 4.1. Two improvements were added to the algorithm, the first, referred to as probability oscillation, provides additional diversity during solution perturbation. Figure 7 details these changes to the perturbation portion of the algorithm previously described in Figure 4.

**Figure 7** Probability oscillation

```
1. Inputs:  x, n, DOE_estimate, DOE_sort_order
2.   Output:  x
3.   for ( i = 0 to n ) {
4.   j = DOE_sort_order ( i )
5.       if (GetRandomDouble(0,1) > 0.5)
6.            prob = UseDistribution1( DOE_estimate )
7.       else prob  = UseDistribution2( DOE_estimate )
8.       if ( prob > Get_random_float (0,1) ) k = 1 else k = 0
9.       if (GetRandomDouble(0,1) > 0.5)
10.           SetAspirationLevelLow (aspiration_level)
11.      else SetAspirationLevelHigh (aspiration_level)
12.      if ( x[ j ] != kAND Delta_objective( x [ j ] ) > aspiration_level )
13.              x[ j ] = k  // make the change
14.   }  // end for all variables i in x
```

**Figure 8** The two probability sampling distributions (see online version for colours)



The results obtained for the larger set of problems were affected by the distribution used to provide the probability of perturbing an $x_i$ from zero to one. Figure 8 illustrates the shape of the sampling distribution used in lines 6 and 7 of Figure 7. Distribution 1 corresponds to a normal distribution and indicates that the probability that a variable $x_i$ will be set to 1 is near zero for $x_i$ with DOE estimates that are less than three standard deviations from the mean DOE estimate, and that the probability that variable $x_i$ will be set to 1 is close to one for $x_i$ with DOE estimates that are greater than three standard deviations greater than the mean. Distribution 2 serves to provide diversification from the settings expected.

The second improvement made was in the path relinking module. By simply adding a call to the local search routine after line 6 in Figure 6, the search is allowed off the direct path from the starting to the ending solution. For n difference bits, there are n! different paths between the two solutions, so the path relinking problem is difficult and this heuristic, referred to as path loitering, worked well in conjunction with probability oscillation in quickly finding solutions. Together, the improved algorithm with DOE preprocessing of Q, probability oscillation and path loitering was termed POPL.

## 4 Computational results

All problems used a Dell computer with 12 GB RAM and 3.4 GHz 4-core hyper-threaded i7-2600 processor running Windows 7. Cplex took advantage of the 8 virtual cores, however the approaches compiled and studied here only utilised single threads. The DOE presolve is very amenable to parallel implementations because each test run in the DOE is independent. Thus, a DOE needing 16,000 test runs that takes 60 seconds using a single thread, could expect dramatic improvements in time with a paralleled hyper-threading implementation.

The four initial approaches used to verify the benefits of preprocessing have been discussed in Section 3 and are compared using two sets of commonly available benchmark problems. The first set of problems is from the ORLIB (Beasley, 2014) collection and consists of ten sparse instances (density = 0.1), all of size 2,500 variables. The differences of the four approaches from the best reported solutions and times is provided in Table 1 along with a comparison to the times reported by Wang et al. (2012). The termination criteria yielding the times reported for the four approaches was 20,000 iterations through the main loop.

Of the four approaches, the two using preprocessing, DG and DGR, performed on average much better than the two approaches without preprocessing, G and GR. DG and DGR average gap from the best reported solution was less than 0.003% as compared to about 0.75% for G and GR. The approach G was very fast to converge, but has no mechanism to break out of a local optimality trap. GR, with simple path relinking, was on average the slowest, slower than DGR, possibly because of the larger number of difference bits to be explored between the best solutions and the working solution in every iteration. By finding the best reported solution to all ten problems in the second best time, approach DG provided the best mix of solution quality and time to solution.

As an additional point of reference, these problems were also solved via Cplex v12, using a quadratic model. Cplex always found a solution within 1% of the best reported in about 60 seconds, but rarely found a major improvement in the objective or lower bound, and was terminated after one hour with an average 16% MIP gap.

**Table 1** ORLIB benchmark results

| | | *Times (sec)** | | | | | *Percent gap from best reported* | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| *ORLIB #* | *Best reported solution* | *DGR* | *DG* | *G* | *GR* | *PR2* | *Cplex* | *DGR* | *DG* | *G* | *GR* |
| 1 | 1515944 | 1 | 0.6 | 4 | 7 | 11 | 0.83% | 0.00% | 0.00% | 1.20% | 1.13% |
| 2 | 1471392 | 94 | 42 | 4 | 168 | 101 | 0.70% | 0.03% | 0.00% | 0.75% | 0.69% |
| 3 | 1414192 | 3 | 44 | 4 | 92 | 49 | 0.62% | 0.00% | 0.00% | 0.82% | 0.81% |
| 4 | 1507701 | 0.4 | 0.3 | 3 | 89 | 6 | 0.50% | 0.00% | 0.00% | 0.84% | 0.71% |
| 5 | 1491816 | 1 | 1 | 1 | 162 | 14 | 0.54% | 0.00% | 0.00% | 0.66% | 0.41% |
| 6 | 1469162 | 8 | 5 | 3 | 103 | 25 | 0.57% | 0.00% | 0.00% | 0.90% | 0.64% |
| 7 | 1479040 | 2 | 26 | 4 | 162 | 48 | 0.83% | 0.00% | 0.00% | 0.77% | 0.71% |
| 8 | 1484199 | 1 | 19 | 5 | 27 | 20 | 0.48% | 0.00% | 0.00% | 0.74% | 0.40% |
| 9 | 1482413 | 17 | 0.5 | 4 | 7 | 51 | 0.55% | 0.00% | 0.00% | 0.55% | 0.50% |
| 10 | 1483355 | 129 | 52 | 5 | 147 | 55 | 0.58% | 0.00% | 0.00% | 0.68% | 0.57% |
| | Averages | 26 | 19 | 4 | 96 | 38 | 0.62% | 0.003% | 0.00% | 0.79% | 0.66% |

Twenty-one dense Q instances, size 3,000 to 7,000 variables are provided by the Palubeckis (2006) generator and are well-studied and challenging. These problems range in density from 50% (problems ending in .1), to 80% (problems ending in .2 and .3) and 100%. DG, preprocessing with no relinking, was generally faster than DGR with relinking, and was on average closer to the best reported, although both approaches are on average within 99.9% of the best reported. Comparing DG times to those reported in Wang et al. (2012), DG performs about the same overall for the 50% dense instances and was about 4x faster for the 80% to 100% dense problems, although time-to-solution comparisons can be tenuous, e.g., the time to a solution that is within .01% of optimal may be considerably less than the time to find the optimal solution.

**Table 2** Palubeckis' benchmark results

| | | *Times (sec)* | | | | | *% diff from best reported* | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| *Problem #* | *Best reported* | *DGR* | *DG* | *G* | *GR* | *PR2* | *DGR* | *DG* | *G* | *GR* |
| p3000.1 | 3931583 | 379 | 85 | 9 | 225 | 81 | 0.000% | 0.000% | 1.40% | 1.05% |
| p3000.2 | 5193073 | 4 | 18 | 440 | 239 | 64 | 0.000% | 0.000% | 1.04% | 0.81% |
| p3000.3 | 5111533 | 18 | 71 | 8 | 80 | 155 | 0.000% | 0.014% | 0.89% | 0.92% |
| p3000.4 | 5761822 | 52 | 29 | 8 | 44 | 97 | 0.000% | 0.000% | 1.10% | 1.12% |
| p3000.5 | 5675625 | 19 | 57 | 6 | 27 | 226 | 0.000% | 0.000% | 0.72% | 0.92% |
| p4000.1 | 6181830 | 60 | 27 | 14 | 107 | 159 | 0.000% | 0.000% | 0.64% | 0.38% |
| p4000.2 | 7801355 | 127 | 37 | 19 | 273 | 302 | 0.027% | 0.000% | 0.47% | 0.73% |
| p4000.3 | 7741685 | 40 | 35 | 15 | 289 | 436 | 0.001% | 0.000% | 0.53% | 0.69% |
| p4000.4 | 8711822 | 143 | 17 | 15 | 16 | 392 | 0.000% | 0.000% | 0.65% | 0.00% |
| p4000.5 | 8908979 | 13 | 70 | 17 | 19 | 327 | 0.000% | 0.000% | 0.83% | 0.79% |
| p5000.1 | 8559680 | 1133 | 736 | 35 | 43 | 387 | 0.021% | 0.004% | 0.88% | 0.85% |
| p5000.2 | 10836019 | 330 | 797 | 38 | 635 | 609 | 0.006% | 0.006% | 0.75% | 0.59% |

**Table 2** Palubeckis' benchmark results (continued)

| | | *Times (sec)* | | | | | *% diff from best reported* | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| *Problem #* | *Best reported* | *DGR* | *DG* | *G* | *GR* | *PR2* | *DGR* | *DG* | *G* | *GR* |
| p5000.3 | 10489137 | 606 | 17 | 31 | 536 | 967 | 0.048% | 0.004% | 0.92% | 0.55% |
| p5000.4 | 12252318 | 163 | 183 | 36 | 474 | 767 | 0.030% | 0.005% | 0.85% | 0.85% |
| p5000.5 | 12731803 | 252 | 99 | 23 | 410 | 726 | 0.000% | 0.008% | 0.71% | 0.78% |
| p6000.1 | 11384976 | 83 | 1580 | 67 | 1022 | 1136 | 0.050% | 0.034% | 0.85% | 1.02% |
| p6000.2 | 14333855 | 837 | 226 | 56 | 888 | 1076 | 0.037% | 0.012% | 0.97% | 0.92% |
| p6000.3 | 16132915 | 1432 | 1330 | 64 | 1060 | 1053 | 0.032% | 0.000% | 0.88% | 1.09% |
| p7000.1 | 14478676 | 250 | 1407 | 6 | 88 | 1917 | 0.004% | 0.003% | 0.34% | 0.54% |
| p7000.2 | 18249948 | 954 | 624 | 1 | 116 | 1591 | 0.001% | 0.001% | 0.45% | 0.73% |
| p7000.3 | 20446407 | 39 | 152 | 3 | 151 | 1503 | 0.028% | 0.036% | 0.62% | 0.04% |
| | Averages | 330 | 362 | 43 | 321 | 665 | 0.014% | 0.006% | 0.79% | 0.73% |

## *4.1 Larger, diverse problem set*

The previous benchmark problems are either sparse or dense, if we say that a dense problem is more than 50% non-zeros in the Q matrix. The problem generator provided by Palubeckis (2006) for the problems in Table 2 creates diagonal and non-diagonal elements of the Q matrix with all $Q_{ij}$ having equal probability of being drawn from a uniform distribution. Thus, there will generally be no outliers, particularly for the dense problems and there will be no central tendency of the data. However, outliers and clustering are common tendencies seen in distributions of real data, in other words, using benchmark problems with uniform distributions of data can create challenging, but perhaps unrealistic problems.

With improvements to computing power and memory, as well as software improvements, the problem size associated with UBQP has been growing, so that these two benchmarks are becoming easier to solve. In fact, in this research, a basic approach worked well on smaller uniformly distributed problems, but did not scale well to large, non-uniformly distributed problems, as will be described next.

Therefore, a new set of larger problems, based on a simple modification to the code provided by Palubeckis (2006), was created with non-uniform magnitude distributions and different ranges for the diagonal and non-diagonal elements. Having one bound for both the linear and quadratic elements of Q can lead to diagonal dominance, which creates very simple to solve UBQP. Table 3 illustrates the differences in the ranges used for the uniformly generated problems of Tables 2 and 3 and the larger non-uniform problems generated.

**Table 3** Example changes to problem generator input file for the larger problems tested

| *Generator type* | *lower bound linear* | *upper bound linear* | *lower bound quadratic* | *upper bound quadratic* | *Modularity factor* |
|---|---|---|---|---|---|
| Uniform (Palubeckis, 2006) | –100 | 100 | –100 | 100 | n/a |
| Non-uniform | –50 | 50 | –150 | 50 | 15 |

In addition to the changes in the linear and quadratic lower and upper bounds, occasional linear (diagonal) element outliers were created by decreasing the random element by the size of the linear upper bound. Quadratic outliers were created by occasionally increasing a random element by the quadratic upper bound. How often these modifications were made is determined by the modularity factor that is input to the generator. A modularity factor of 15 would generate a possible outlier every fifteenth element. Note that this implementation does not guarantee an outlier will be created.

The problem generator code provided by Palubeckis (2006) was only very slightly modified and is shown in Figure 9. Lines 5 and 9 of Figure 9 shows how the MOD factor, based on using the remainder function (MOD or %), is used to select an element of Q for adjustment. The *higher* the MOD factor, the *less* often an element may be placed outside the limits. (For the larger problems with no outliers, the original Palubeckis code was used.) The desired effect was to create more realistic problem instances as well as to place some outlier elements for the DOE to discover, thus influencing the search portion of the algorithm. This modification is available from the first author's website (Lewis, 2013), or via e-mail, along with the input files needed to replicate the problems.

**Figure 9** Pseudo-code for non-uniformly distributed Q

```
1. Inputs: MOD_factor, density, lower_lin, upper_lin, lower_quad, upper_quad
2. Output:  Q
3. for ( i = 0 to size)
4.    Q[i][i] = GetRandomInt(lower_lin, upper_lin)
5.    if ( i % MOD_factor == 0 ) Q[i][i] = Q[i][i] - upper_lin
6. for ( j = i to size)
7.       if ( GetRandomFloat(0,1) < density )
8.          Q[i][i] = GetRandomInt(lower_quad, upper_quad)
9.          if ( (i+j) % MOD_factor == 0 ) Q[i][j] = Q[i][j] + upper_quad
```

The larger suite of test problems was created to investigate the effects on problem difficulty of changes in problem size (9,000 to 15,000 variables), density (15% and 50%) and modularity (four levels of uniformity). The results shown in Table 4 are for the 15% problems and are sorted by modularity factor and problem size. For example, problem ID LK9000.15.0 is a 9,000 variable problem that is 15% dense and all variables are drawn from a uniform distribution and not modified in any way. Table 4 presents the results on the 50% dense problems. Tables 4 and 5 compare the results of the POPL algorithm with the executable of the path relinking code (PR2) from Wang et al. (2012) in time to solution and solution quality. Thus, for LK9000.15.0, the solution of PR2 was about 99% that of POPL, but PR2 was about 60% faster.

Focusing on solution quality, Table 4 shows that POPL and PR2 differ the most on the uniformly distributed problems and the MOD factor 60 problems (.0 and .60). The .60 problems will have the fewest number of Q elements placed outside of the input bounds. For the MOD factor 15 and 30 problems (.15 and .30), POPL and PR2 often had the same solution values, although POPL was generally faster. Surprisingly, POPL performed well on the MOD factor 0 (uniform distribution) problems on three of the 15.0 problems and all of the 50.0 problems (see Table 5).

The 15.60 problems were the most difficult for POPL, possibly because the preprocessing could not reliably differentiate the few outlier elements against the larger backdrop of the majority uniform elements. POPL solutions were about 13% lower than

PR2 for the 15.60 problems. These problems seem to have 'hidden' solutions that are difficult to find. For example, POPL found a much better solution for problem LK1000.15.60, but much worse for problems LK11000.15.60 and LK15000.15.60. Based on this, we expect that problems LK9000.15.60, LK12000.15.60 and LK13000.15.60 have much better solutions than what either POPL or PR2 found.

**Table 4** Results for the 15% dense new set of problems

| *Problem ID* | *POPL values* | *PR2 values* | *POPL time* | *PR2 times* | *Solution PR2/POPL* | *Time PR2/POPL* |
|---|---|---|---|---|---|---|
| LK9000.15.0 | 21899 | 21621 | 1110 | 662 | 0.99 | 0.6 |
| LK10000.15.0 | 22644 | 22230 | 459 | 904 | 0.98 | 2.0 |
| LK11000.15.0 | 21489 | 22171 | 198 | 1968 | 1.03 | 9.9 |
| LK12000.15.0 | 22585 | 23572 | 296 | 444 | 1.04 | 1.5 |
| LK13000.15.0 | 23758 | 23287 | 139 | 1391 | 0.98 | 10.0 |
| LK14000.15.0 | 23654 | 24580 | 1192 | 2159 | 1.04 | 1.8 |
| LK15000.15.0 | 24323 | 24428 | 1086 | 3043 | 1.00 | 2.8 |
| LK9000.15.15 | 188728 | 188728 | 827 | 162 | 1.00 | 0.2 |
| LK10000.15.15 | 224097 | 224097 | 248 | 111 | 1.00 | 0.4 |
| LK11000.15.15 | 257485 | 257485 | 55 | 202 | 1.00 | 3.7 |
| LK12000.15.15 | 282700 | 282700 | 290 | 148 | 1.00 | 0.5 |
| LK13000.15.15 | 352213 | 352213 | 128 | 167 | 1.00 | 1.3 |
| LK14000.15.15 | 394943 | 394943 | 80 | 34 | 1.00 | 0.4 |
| LK15000.15.15 | 416984 | 417021 | 486 | 689 | 1.00 | 1.4 |
| LK9000.15.30 | 67368 | 68763 | 123 | 87 | 1.02 | 0.7 |
| LK10000.15.30 | 89985 | 89985 | 387 | 139 | 1.00 | 0.4 |
| LK11000.15.30 | 89530 | 89530 | 1197 | 956 | 1.00 | 0.8 |
| LK12000.15.30 | 104756 | 106135 | 305 | 901 | 1.01 | 3.0 |
| LK13000.15.30 | 124451 | 124451 | 758 | 258 | 1.00 | 0.3 |
| LK14000.15.30 | 139844 | 139844 | 477 | 133 | 1.00 | 0.3 |
| LK15000.15.30 | 153897 | 148954 | 3782 | 1383 | 0.97 | 0.4 |
| LK9000.15.60 | 24156 | 24408 | 981 | 1170 | 1.01 | 1.2 |
| LK10000.15.60 | 36477 | 24892 | 100 | 820 | 0.68 | 8.2 |
| LK11000.15.60 | 25375 | 32476 | 908 | 2627 | 1.28 | 2.9 |
| LK12000.15.60 | 25310 | 26187 | 377 | 547 | 1.03 | 1.5 |
| LK13000.15.60 | 25766 | 26578 | 500 | 1481 | 1.03 | 3.0 |
| LK14000.15.60 | 52481 | 52481 | 1736 | 1983 | 1.00 | 1.1 |
| LK15000.15.60 | 27914 | 52219 | 1624 | 2535 | 1.87 | 1.6 |
| | | | | Averages | 1.03 | 2.2 |

Table 5 presents the results on the larger 50% dense problems. POPL and PR2 are very comparable in solution quality for all the denser problems, except for those that are uniformly distributed, in which case POPL found solutions that were on average 7%

better, and obtained much faster. On all the other dense problems the solutions are much closer than they were for the 15% dense problems. Problem size seemed to affect POPL less than PR2, for example, the 9,000 variable problems took on average about 500 seconds for both approaches, but as problem size increases, POPL times were generally, but not always, less. The average time for the 15,000 variable problems was 1,100 seconds for POPL and 2,051 for PR2.

**Table 5** Results for the 50% dense new set of problems

| *Problem ID* | *POPL values* | *PR2 values* | *POPL time* | *PR2 times* | *Solution PR2/POPL* | *Time PR2/POPL* |
|---|---|---|---|---|---|---|
| LK9000.50.0 | 8662 | 7986 | 79 | 1114 | 0.92 | 14.1 |
| LK10000.50.0 | 8697 | 7867 | 183 | 1049 | 0.90 | 5.7 |
| LK11000.50.0 | 8473 | 7860 | 37 | 2161 | 0.93 | 58.4 |
| LK12000.50.0 | 8974 | 8008 | 63 | 625 | 0.89 | 9.9 |
| LK13000.50.0 | 8890 | 8717 | 107 | 1165 | 0.98 | 10.9 |
| LK14000.50.0 | 9329 | 8715 | 607 | 1950 | 0.93 | 3.2 |
| LK15000.50.0 | 9442 | 8711 | 439 | 2980 | 0.92 | 6.8 |
| LK9000.50.15 | 369653 | 369653 | 33 | 105 | 1.00 | 3.2 |
| LK10000.50.15 | 435391 | 435391 | 30 | 580 | 1.00 | 19.3 |
| LK11000.50.15 | 461673 | 461673 | 44 | 888 | 1.00 | 20.2 |
| LK12000.50.15 | 511274 | 511274 | 104 | 310 | 1.00 | 3.0 |
| LK13000.50.15 | 645267 | 645938 | 194 | 678 | 1.00 | 3.5 |
| LK14000.50.15 | 695877 | 695877 | 197 | 542 | 1.00 | 2.8 |
| LK15000.50.15 | 763238 | 763222 | 325 | 2490 | 1.00 | 7.7 |
| LK9000.50.30 | 129814 | 129814 | 36 | 81 | 1.00 | 2.3 |
| LK10000.50.30 | 151752 | 151752 | 33 | 793 | 1.00 | 24.0 |
| LK11000.50.30 | 180159 | 180159 | 50 | 1801 | 1.00 | 36.0 |
| LK12000.50.30 | 197362 | 197362 | 44 | 152 | 1.00 | 3.5 |
| LK13000.50.30 | 234447 | 238364 | 14 | 87 | 1.02 | 6.2 |
| LK14000.50.30 | 256836 | 256836 | 81 | 852 | 1.00 | 10.5 |
| LK15000.50.30 | 284464 | 284464 | 80 | 751 | 1.00 | 9.4 |
| LK9000.50.60 | 50246 | 50246 | 1179 | 82 | 1.00 | 0.1 |
| LK10000.50.60 | 54190 | 54190 | 425 | 109 | 1.00 | 0.3 |
| LK11000.50.60 | 63788 | 63788 | 45 | 410 | 1.00 | 9.1 |
| LK12000.50.60 | 71639 | 71639 | 44 | 148 | 1.00 | 3.4 |
| LK13000.50.60 | 81060 | 81060 | 73 | 185 | 1.00 | 2.5 |
| LK14000.50.60 | 96929 | 96929 | 129 | 279 | 1.00 | 2.2 |
| LK15000.50.60 | 100774 | 103399 | 985 | 2538 | 1.03 | 2.6 |
| | | | | | 0.98 | 10.0 |

## 5 DOE results analysis

In this section we discuss results associated with the DOE preprocessing and how they provide evidence for the shape of the probability distributions used.

In the figure below, for problem p3000.2 in Table 3, the average estimated effect on the objective function of changing a variable from 0 to 1 was 10, with a standard deviation of 2197. For the best reported value for this problem, all x values in the solution above two standard deviations from the average were set to 1 and 1.4% of $x$ values that were two standard deviations *below* the average were set to 1, in other words 98.6% of variables whose DOE estimate was far below the average were set to 0. Closer to the average effect there is a more uniform distribution of 1 s and 0 s. This pattern was typical and was used to set the 1, 2 and 3 standard deviation limits in solution perturbation.

**Figure 10** Distribution of x-values by DOE estimated effect for best known solution to p3000.2 (see online version for colours)

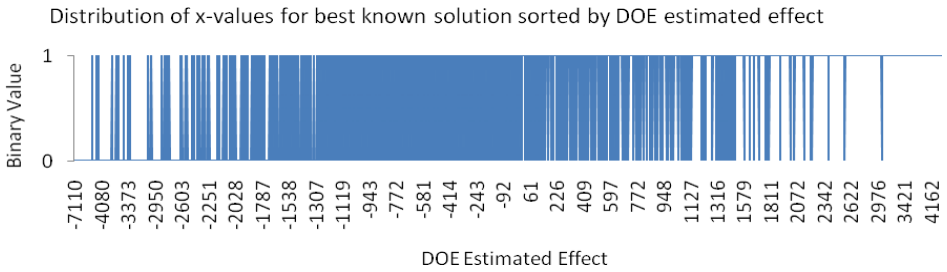
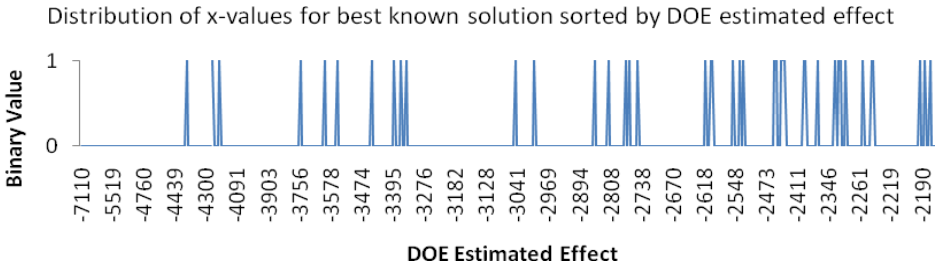


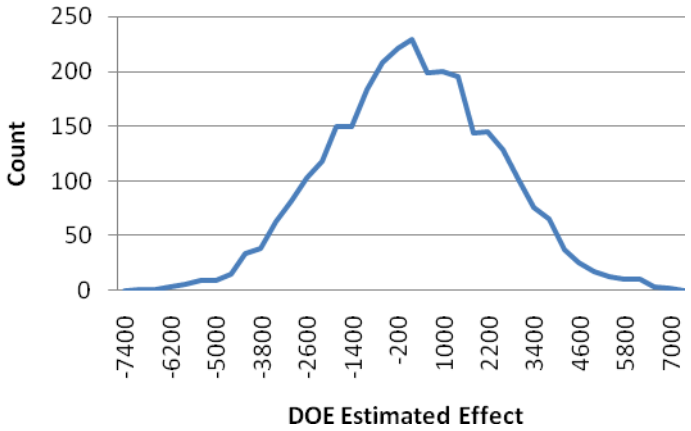

Figure 11 expands the area in Figure 10 to highlight the distribution of solution values greater than one standard deviation below the average measured effect, i.e., DOE estimates $< \mu - \sigma$. The $x_i$ set to 1 are only 1.4% so that 98.6% are set to 0. Thus, in this solution, which generates a very good objective value, the variables whose DOE estimates were well below the average were equal to 0 and of the 58 $x_i$ whose DOE was less than 4,400 $(\mu - 2\sigma)$ no $x_i$ were set to 1. However, since the variables described as outliers in the DOE distribution are estimated to have large effects, one should be careful in making the assumption that they should simply be set to 0 or 1, prior to any search, in the hope of reducing problem size without affecting solution quality.

**Figure 11** Magnified view of distribution of Figure 8 for DOE estimates $< \mu - \sigma$ (see online version for colours)



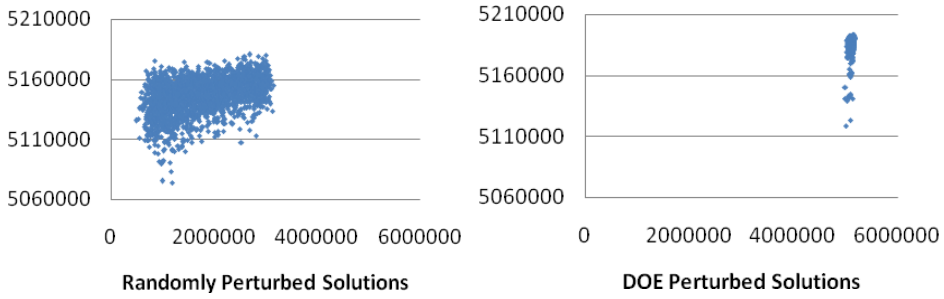
The rationale for borrowing some ideas from statistical methods is based on examining data from the DOE as shown in Figure 12 for problem p3000.1. This distribution has a decidedly normal appearance, and so the probabilities used for generating solutions are based on estimated effects being one, two, or three standard deviations from the average DOE effect. Statistical outliers are those $x_i$ that are more than $3\sigma$ from the mean, i.e., those $x_i$ more than $3\sigma$ from $\mu$ are likely to be set in the final solution to their measured effect. However, this is not a guarantee.

**Figure 12** Frequency distribution of DOE estimated effects for problem P3000.2 (see online version for colours)




The scatter plots of the data from the DG and GR approaches for problem p3000.2 is shown in Figure13. The plots compare the quality of the solutions input to, then output from the `Local_Search` module for the two approaches, one randomly perturbing solutions, the other using the DOE data for perturbation. The points are collected by saving the objective value of the solutions generated by the `Perturb` module and provided to the `Local_Search` module, whose associated output is also saved then paired to the input. Figure 13 shows that the randomly generated starting solutions are more diverse and of lower objective function value than the DOE generated starting solutions. Whereas diversity can be very helpful in breaking out of local optimality, it can slow down the search process and while generating high quality solutions as input to a search can provide good solutions quickly, global optima may not be discovered. Thus, combining aspects from both seems a reasonable approach.

**Figure 13** Plot of objective values for P3000.2 using randomly and DOE generated solutions (see online version for colours)




## 6 Conclusions

In this paper we have described an approach to solving UBQP that is based on local improvement of solutions perturbed according to global probabilities of a variable's positive effect on the objective function value. These probabilities are calculated from an unbiased sampling of the solution space using an experimental design methodology. Algorithms based on using these probabilities are compared to the same approach without this information with the results indicating that preprocessing offers a noticeable advantage. A new set of larger, diverse problems was created to test the approach, and based on comparison with a recent path relinking code, the approach was modified to sample from an oscillating probability distribution. Coupled with an improved path relinking module, the new approach compares well. We also discuss the effects of the distribution of the elements of Q on problem difficulty.

## References

Alidaee, B., Glover, F., Kochenberger, G. and Rego, C. (2005) 'A new modeling and solution approach for the number partitioning problem', *Journal of Applied Mathematics & Decision Sciences*, No. 2, pp.113–121.

Alidaee, B., Kochenberger, G. and Ahmadian, A. (1994) '0-1 Quadratic programming approach for optimum solutions of two scheduling problems', *International Journal of Systems Science*, Vol. 25, No. 2, pp.401–408.

Alidaee, B., Kochenberger, G., Lewis, K., Lewis, M. and Wang, H. (2008) 'A new approach for modeling and solving set packing problems', *European Journal of Operational Research*, Vol. 186, No. 2, pp.504–512.

Amini, M., Alidaee, B. and Kochenberger, G. (1999) 'A scatter search approach to unconstrained quadratic binary programs', in *New Ideas in Optimization*, pp.317–330, McGraw-Hill Ltd., UK.

Barahona, F., Grötschel, M., Jünger, M. and Reinelt, G. (1988) 'An application of combinatorial optimization to statistical physics and circuit layout design', *Operations Research*, Vol. 36, No. 3, pp.493–513.

Beasley, J. (2014) *Welcome to the OR-Library* [online] http://www.people.brunel.ac.uk/~mastjjb/jeb/info.html (accessed March 2014).

Billionnet, A. and Sutter, A. (1994) 'Minimization of a quadratic pseudo-Boolean function', *European Journal of Operational Research*, Vol. 78, No. 1, pp.106–115.

Boros, E. and Hammer, P. (1991) 'The max-cut problem and quadratic 0-1 optimization polyhedral aspects, relaxations and bounds', *Annals of Operations Research*, Vol. 33, No. 4127, pp.151–180.

Boros, E., Hammer, P. and Tavares, G. (2007) 'Local search heuristics for quadratic unconstrained binary optimization (QUBO)', *Journal of Heuristics*, Vol. 13, No. 2, pp.99–132.

Chinchuluum, A., Pardalos, P. and Enkhbat, R. (2005) 'Global minimization algorithms for concave quadratic programming problems', *Optimization*, Vol. 54, No. 6, pp.627–639.

Dinh, T., Canh, N. and Le Thi, H. (2010) 'An efficient combined DCA and B&B using DC/SDP relaxation for globally solving binary quadratic programs', *Journal of Global Optimization*, Vol. 48, No. 4, pp.595–632.

Douiri, S. and Elbernoussi, S. (2012) 'The unconstrained binary quadratic programming for the sum coloring problem', *Modern Applied Science*, Vol. 6, No. 9, pp.26–33.

Glover, F., Alidaee, B., Rego, C. and Kochenberger, G. (2002) 'One-pass heuristics for large-scale unconstrained binary quadratic problems', *European Journal of Operational Research*, Vol. 137, No. 2, pp.272–287.

Glover, F., Kochenberger, G. and Alidaee, B. (1998) 'Adaptive memory tabu search for binary quadratic programs', *Management Science*, Vol. 44, No. 3, pp.336–345.

Glover, F., Lu, Z. and Hao, J. (2010) 'Diversification-driven tabu search for unconstrained binary quadratic problems', *4OR*, Vol. 8, No. 3, pp.239–253.

Gueye, S. and Michelon, P. (2009) 'A linearization framework for unconstrained quadratic (0-1) problems', *Discrete Applied Mathematics*, Vol. 157, No. 6, pp.1255–1266.

Gulati, V., Gupta, S. and Mittal, A. (1984) 'Unconstrained quadratic bivalent programming problem', *European Journal of Operational Research*, Vol. 15, No. 1, pp.121–125.

Hansen, P., Jaumard, B. and Mathon, V. (1993) 'State-of-the-art survey – constrained nonlinear 0-1 programming', *ORSA Journal on Computing*, Vol. 5, No. 2, pp.97–119.

Huang, H., Pardalos, P. and Prokopyev, O. (2006) 'Lower bound improvement and forcing rule for quadratic binary programming', *Computational Optimization and Applications*, Vol. 33, No. 4130, pp.187–208.

Kochenberger, G., Glover, F. and Lewis, K. (2005a) 'Using the unconstrained quadratic program to model and solve Max 2-SAT problems', *International Journal of Operational Research*, Vol. 1, No. 1, pp.89–100.

Kochenberger, G., Glover, F., Alidaee, B. and Rego, C. (2004) 'A unified modeling and solution framework for combinatorial optimization problems', *OR Spectrum*, Vol. 26, No. 3, pp.237–250.

Kochenberger, G., Glover, F., Alidaee, B. and Wang, H. (2005b) 'Clustering of microarray data via clique partitioning', *Journal of Combinatorial Optimization*, Vol. 10, No. 1, pp.77–92.

Kochenberger, G., Hao, J., Lu, Z., Wang, H. and Glover, F. (2013) 'Solving large scale max cut problems via tabu search', *Journal of Heuristics*, Vol. 19, No. 4, pp.565–571.

Lewis, M. (2008) 'On the use of guided design search for discovering significant decisions variables in the fixed charge capacitated multicommodity network design problem', *Networks*, Vol. 53, No. 1, pp.6–18.

Lewis, M. (2013) *Large Difficult xQx Instances* [online] http://www.academic.missouriwestern.edu/mlewis14/Research/Large_UBQP/Large%20difficult%20Q.htm (accessed March 2014).

Lewis, M., Alidaee, B. and Kochenberger, G. (2005) 'Using xQx to model and solve the uncapacitated task allocation problem', *Operations Research Letters*, Vol. 33, No. 2, pp.176–182.

Lewis, M., Alidaee, B. and Kochenberger, G. (2009) 'A note on xQx as a modelling and solution framework for the linear ordering problem', *International Journal of Operational Research*, Vol. 5, No. 2, pp.152–162.

Lewis, M., Kochenberger, G. and Alidaee, B. (2008) 'A new modeling and solution approach for the set-partitioning problem', *Computers & Operations Research*, Vol. 35, No. 3, pp.807–813.

Lewis, M., Lewis, K. and White, B.J. (2006) 'Guided design search in the interval-bounded sailor assignment problem', *Computers and Operations Research*, Vol. 33, No. 6, pp.1664–1680.

Lewis, M., Wang, H. and Kochenberger, G. (2104) 'Exact solutions to the capacitated clustering problem: a comparison of two models', *Annals of Data Science*, Vol. 1, No. 1, pp.1–9.

Lu, Z., Glover, F. and Hao, J-K. (2010a) 'A hybrid metaheuristic approach to solving the UBQP problem', *European Journal of Operational Research*, Vol. 207, No. 3, pp.1254–1262.

Lu, Z., Hao, J. and Glover, F. (2010b) 'A study of memetic search with multi-parent combination for UBQP', *Evolutionary Computation in Combinatorial Optimization*, *10th European Conference, EvoCOP 2010, Proceedings*, Istanbul, Turkey, 7–9 April 2010, Vol. 6022, pp.154–165, Springer Berlin Heidelberg.

Mauri, G. and Lorena, L. (2012) 'Improving a Lagrangian decomposition for the unconstrained binary quadratic programming problem', *Computers & Operations Research*, Vol. 39, No. 7, pp.1577–1581.

Merz, P. (2004) 'Advanced fitness landscape analysis and the performance of memetic algorithms', *Evolutionary Computation*, Vol. 12, No. 3, pp.303–325.

Merz, P. and Freisleben, B. (2002) 'Greedy and local search heuristics for unconstrained binary quadratic programming', *Journal of Heuristics*, Vol. 8, No. 2, pp.197–213.

Palubeckis, G. (1995) 'A heuristic-based branch and bound algorithm for unconstrained quadratic zero-one programming', *Computing*, Vol. 54, No. 4, pp.283–301.

Palubeckis, G. (2004) 'Multistart tabu search strategies for the unconstrained binary quadratic optimization problem', *Annals of Operations Research*, Vol. 131, No. 4127, pp.259–282.

Palubeckis, G. (2006) 'Iterated tabu search for the unconstrained binary quadratic optimization problem', *Informatica*, Vol. 17, No. 2, pp.279–296.

Pan, S., Tan, T. and Jiang, Y. (2008) 'A global continuation algorithm for solving binary quadratic programming problems', *Computational Optimization & Applications*, Vol. 41, No. 3, pp.349–362.

Pardalos, P. and Rodgers, G. (1990) 'Computational aspects of a branch and bound algorithm for quadratic zero-one programming', *Computing*, Vol. 45, No. 2, pp.131–144.

Pardalos, P. and Xue, J. (1994) 'The maximum clique problem', *Journal of global Optimization*, Vol. 4, No. 3, pp.301–328.

Wang, H., Alidaee, B., Glover, F. and Kochenberger, G. (2006) 'Solving group technology problems via clique partitioning', *International Journal of Flexible Manufacturing Systems*, Vol. 18, No. 2, pp.77–97.

Wang, Y., Lu, Z., Glover, F. and Hao, J. (2012) 'Path relinking for unconstrained binary quadratic programming', *European Journal of Operational Research*, Vol. 223, No. 3, pp.595–604.

Wang, Y., Lu, Z., Glover, F. and Hao, J-K. (2013a) 'Backbone guided tabu search for solving the UBQP problem', *Journal of Heuristics*, Vol. 19, No. 4, pp.679–695.

Wang, Y., Lu, Z., Glover, F. and Hao, J-K. (2013b) 'Probabilistic GRASP-tabu search algorithms for the UBQP problem', *Computers& Operations Research*, Vol. 40, No. 12, pp.3100–3107.

## Appendix

The detailed steps for creating large experimental designs used to estimate the average effects of flipping binary decision variables from 0 to 1 are as follows.

1 Determine the size (number of rows $n$ by number of columns $m$) of the experimental design **T** according to $m$ = number of binary variable. Note that $n > m$, i.e., the

number of tests to run (or solutions to generate) is always larger than the number of variables.

The number of rows in **T** is the number of test runs and is calculated as $n = 2^{\lceil \log_2(m+epsilon) \rceil}$. The *epsilon* is needed for the case when $m$ is a power of 2 (e.g., $m = 8$ uses $n = 16$ test runs, but $m = 7$ uses $n = 8$ test runs.

2 Generate the $n$ by $m$ fractional factorial table, **T** where $\mathbf{T}_{ij} = \{0,1\}$. These are the diverse set of solutions that will generate xQx values used to estimate the average effect of flipping an $x_i$ from 0 to 1.

To generate a fractional factorial table it is split into two parts, the main effects and the interaction effects (to which any additional variables are assigned). There are $\log_2(n)$ main effect variables. For example, if there are $m = 7$ binary variables, then $n = 8$ and the number of main effect variables is $\log_2(8) = 3$. The settings for the first three main effect columns of $\mathbf{T}_{ij}$ are determined by computing all permutations of the three main effect variable values, i.e., 000, 001, 010, 011 and so on. The remaining interaction columns of $\mathbf{T}_{ij}$ are all combinations of the main effect variable settings, computed by performing logical not-XOR operations, denoted $\overline{\otimes}$, on the corresponding main effect settings.

Continuing with the above eight rows by seven variables example, $x_4$ is assigned the 4th column of **T** and is determined by the interaction effect $x_1 x_2$ and then $x_5$ is assigned the 5th column and is determined by the interaction effect $x_1x_3$ and the 6th column $x_2x_3$ and the 7th column $x_1x_2x_3$. Compute the $\mathbf{T}_{ij}$ setting for the interaction variables using not-XOR operations, e.g., for $x_1 = x_2 = 0$ the binary setting for $x_1 x_2$ is $0 \ \overline{\otimes} \ 0 = 1$. The setting for $x_7$ is determined by $x_1 x_2 x_3$ so that when $x_1 = x_2 = x_3 = 0$ then $0 \ \overline{\otimes} \ 0 \ \overline{\otimes} \ 0 = 0$.

3 For each row $i$ in **T**

Assign values to each of the $x_j$ variables according to row $i$, column $j$ in table **T**, creating the vector x

Calculate xQx and record the value of the objective function

This can be repeated using the *mirror* of **T**, i.e., simply switching all 0s and 1s

4 Compute DOE_avg[i] = average effect of switching a variable $x_i$ from 0 to 1 by computing the average value of xQx for all $x_i$ when $x_i = 0$ and when $x_i = 1$ and subtracting. The overall average effect of switching any variable from 0 to 1 can now be calculated, along with a standard deviation.